PEMROGRAMAN BERORIENTASI OBJEK 2026

Polimorfisme

Satu perintah, banyak bentuk: setiap objek menjawab dengan caranya sendiri

PERTEMUAN

09

dari 16

Program Studi Informatika · Universitas Mulawarman
Jalur praktik: Python dan Java

Tujuan Pembelajaran

Setelah pertemuan ini, kamu mampu:

1. Menjelaskan arti **polimorfisme** dengan analogi sehari-hari dan mengenali wujudnya di dalam kode.
2. Menjelaskan cara kerja polimorfisme: *override* + ***dynamic dispatch***, termasuk beda **tipe variabel** dan **tipe objek**.
3. Merancang **koleksi polimorfik** sehingga jenis baru bisa ditambahkan tanpa mengubah kode pemanggil.
4. Membedakan ***upcasting*** dan ***downcasting***, serta tahu kapan jenis objek perlu diperiksa dengan `isinstance`/`instanceof`.
5. Menjelaskan ***duck typing*** di Python dan alasan Java membutuhkan tipe bersama.
6. Membedakan *overriding* dan *overloading*, serta memakai pengganti *overloading* di Python.

Mengingat Kembali



Dari pertemuan sebelumnya

- **Pewarisan**: kelas anak mewarisi atribut dan *method* induk. `Makanan` **adalah** `Menu` (*is-a*).
- Kelas anak boleh meng-***override*** *method* induk: nama dan parameter sama, isinya ditulis ulang.
- Objek anak juga terhitung objek induk; diperiksa dengan `isinstance` (Python) atau `instanceof` (Java).
- Teka-teki Pertemuan 07: menu dicetak tanpa tahu jenisnya, tetapi makanan dan minuman tampil berbeda. Mengapa?

Agenda Hari Ini

1. Satu perintah, banyak bentuk
2. Cara kerja: *override* dan *dynamic dispatch*
3. Koleksi polimorfik dan `self`/`this`
4. *Upcasting*, *downcasting*, dan *duck typing*
5. *Overloading* dan penggantinya di Python
6. Studi kasus KedaiKita: banyak cara bayar

BAGIAN 1

1

Satu Perintah, Banyak Bentuk

Dari kasir yang kebingungan menuju satu perintah untuk semua jenis pembayaran.

Masalah: Kasir dengan Banyak Cara Bayar

Kejadian	Akibatnya, pada kode berpercabangan
Kedai ingin menerima *voucher*	Fungsi `bayar` harus **dibongkar** untuk menambah satu cabang lagi
Struk, laporan harian, dan biaya layanan juga butuh tahu jenis pembayaran	Percabangan **tersebar**; lupa mengubah satu tempat, hasil tidak konsisten
Ada yang menulis `"Tunai"` dengan huruf T besar	Tidak ada cabang yang cocok; kesalahan baru ketahuan saat program berjalan
Aturan QRIS diubah	Fungsi berisi aturan tunai dan kartu ikut disentuh; bagian lain bisa rusak

Akar masalahnya: **pemanggil** dipaksa tahu semua jenis. Seharusnya ia cukup berkata “bayar!”, lalu setiap jenis mengurus dirinya.

Satu Tombol “Bayar”, Banyak Perilaku

Analogi

- Di aplikasi pesan-antar makanan, kamu selalu menekan tombol **Bayar** yang sama.
- Tunai: pengemudi menerima uang. *E-wallet*: saldo terpotong. Kartu kredit: aplikasi menghubungi bank.
- Tombolnya satu, perilakunya banyak. Tombol putar di ponselmu juga bisa memutar lagu, video, atau *podcast*.

Aplikasi pesan-antar	PBO
Tombol **Bayar** yang selalu sama	Pemanggilan *method* yang sama
Tunai, *e-wallet*, kartu kredit	Objek dari kelas yang berbeda
Yang terjadi di balik tombol	Versi *method* milik setiap objek

Polimorfisme: “Banyak Bentuk”

Polimorfisme (*polymorphism*)

Kemampuan objek-objek dari kelas yang berbeda untuk menanggapi pemanggilan *method* yang sama dengan **cara masing-masing**. Pemanggil cukup tahu *apa* yang diminta, tanpa perlu tahu *bagaimana* setiap objek mengerjakannya.

- Dari bahasa Yunani: *poly* berarti “banyak” dan *morph* berarti “bentuk”.
- Sudah lama kamu pakai: `2 + 3` berarti menjumlahkan, sedangkan `"Kopi" + "Susu"` berarti menyambung.
- Kasir tidak lagi bertanya “kamu jenis apa?”, tetapi cukup mengirim pesan `proses(total)`.

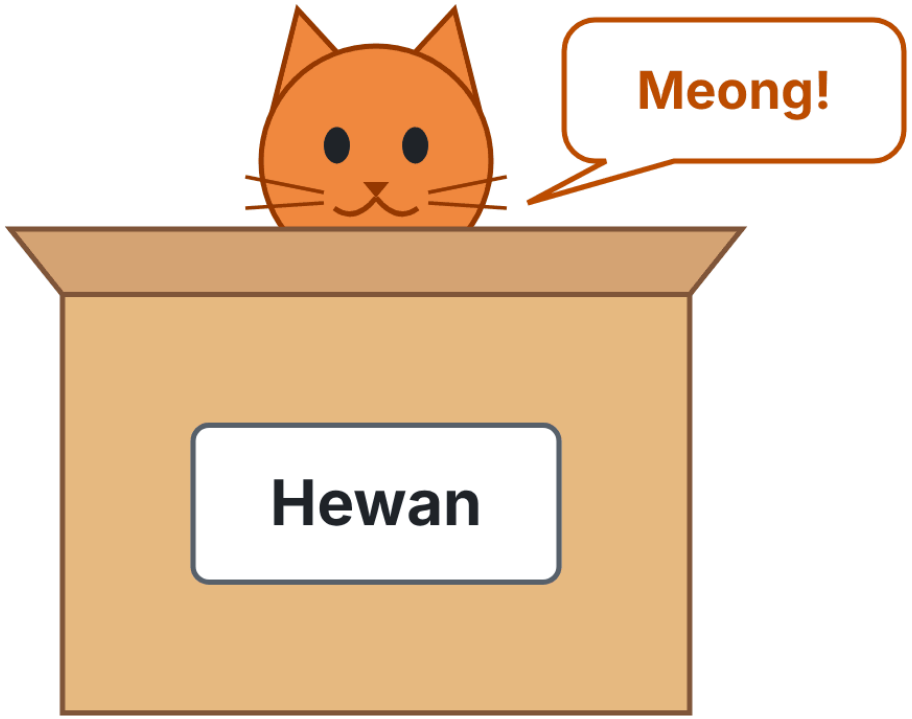
BAGIAN 2

2

Cara Kerja: *Override* dan *Dynamic Dispatch*

Versi *method* yang berjalan dipilih saat program berjalan, sesuai objek yang sebenarnya.

Tipe Variabel dan Tipe Objek



Label kardus = tipe variabel

```
Hewan hewan = new Kucing("Oyen");
```

(di Python: hewan = Kucing("Oyen"))

Isi kardus = tipe objek: Kucing

ditentukan saat objek dibuat, tidak pernah berubah

Disuruh `bersuara()`?

Yang menjawab adalah isinya, jadi hasilnya "Meong!", bukan suara hewan umum.

Label kardus adalah tipe variabel, isinya adalah tipe objek. Untuk *method* yang di-*override*, yang menentukan adalah tipe objek.

Satu Variabel, Berganti Objek

Baris `hewan.bersuara()` ditulis persis sama dua kali, tetapi hasilnya berbeda: `Meong!` lalu `Guk guk!`.

Python python/contoh/contoh01_satu_perintah_banyak_bentuk.py

```python
class Kucing(Hewan):
    def bersuara(self) -> str:
        return "Meong!"

# ... di dalam main()
hewan = Kucing("Oyen")
print(hewan.bersuara())
hewan = Anjing("Bruno")
print(hewan.bersuara())
```

Java java/contoh/Contoh01SatuPerintahBanyakBentuk.java

```java
class Kucing extends Hewan {
    // ... (dilewati)
    @Override
    public String bersuara() {
        return "Meong!";
    }
}
// ... di dalam main
Hewan hewan = new Kucing("Oyen");
System.out.println(hewan.bersuara());
hewan = new Anjing("Bruno");
System.out.println(hewan.bersuara());
```

BAGIAN 3

3

Koleksi Polimorfik dan Polimorfisme dari dalam Kelas

Satu perulangan untuk banyak jenis objek, dan `self/this` yang selalu menunjuk objek sebenarnya.

Jenis Baru tanpa Mengubah Pemanggil

Fungsi laporan tidak menyebut satu pun kelas anak. `Trapesium` ditambahkan belakangan dan langsung ikut dihitung.

Python python/contoh/contoh02_koleksi_polimorfik.py

```python
def cetak_laporan(daftar: list[BangunDatar]):
    total = 0
    for bangun in daftar:
        luas = bangun.luas()
        print(f"{bangun.nama():<30} luas {luas:>4} cm2")
        total += luas
    print(f"Total luas: {total} cm2")
# ... di dalam main()
daftar = [Persegi(10), PersegiPanjang(12, 5), Segitiga(8, 6)]
# ... (dilewati)
daftar.append(Trapesium(6, 10, 4))
daftar.append(BangunDatar())
cetak_laporan(daftar)
```

Java java/contoh/Contoh02KoleksiPolimorfik.java

```java
static void cetakLaporan(List<BangunDatar> daftar) {
    int total = 0;
    for (BangunDatar bangun : daftar) {
        int luas = bangun.luas();
        // ... (dilewati)
        total += luas;
    }
    System.out.println("Total luas: " + total + " cm2");
}
// ... di dalam main
List<BangunDatar> daftar = new ArrayList<>();
// ... daftar.add: Persegi, PersegiPanjang, Segitiga
daftar.add(new Trapesium(6, 10, 4));
daftar.add(new BangunDatar());
cetakLaporan(daftar);
```

Kerangka di Induk, Isian di Anak

`tulis()` hanya ditulis sekali di `KartuUcapan`. `KartuWisuda` cukup meng-*override* `salam()` dan `isi()`.

Python python/contoh/contoh04_polimorfisme_dari_dalam_kelas.py

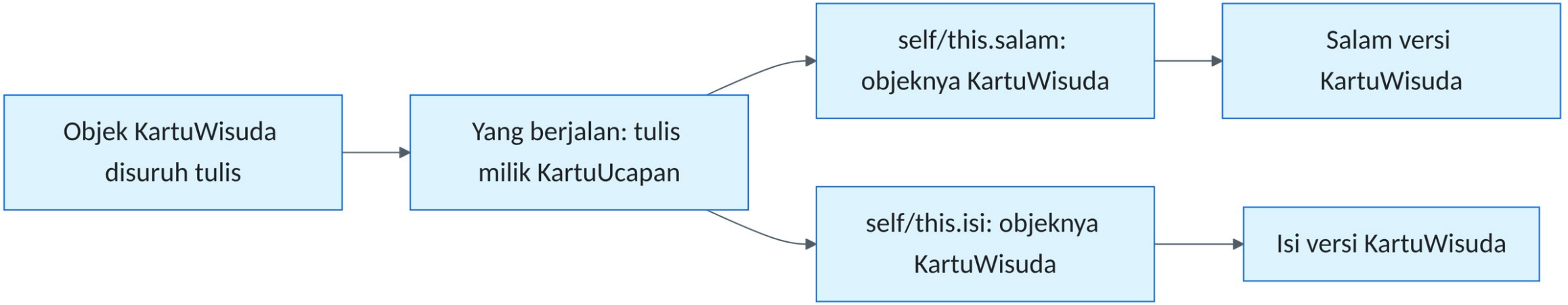
```python
class KartuUcapan:
    # ... (dilewati)
    def tulis(self):
        print("-" * 40)
        print(self.salam())
        print(self.isi())
        # ... (dilewati)

class KartuWisuda(KartuUcapan):
    def salam(self) -> str:
        return f"Selamat, {self._penerima}, S.Kom.!"
    # ... (dilewati)
```

Java java/contoh/Contoh04PolimorfismeDariDalamKelas.java

```java
class KartuUcapan {
    // ... (dilewati)
    public void tulis() {
        System.out.println("-".repeat(40));
        System.out.println(salam());
        System.out.println(isi());
        // ... (dilewati)
    }
}
class KartuWisuda extends KartuUcapan {
    // ... (dilewati)
    @Override
    public String salam() {
        return "Selamat, " + getPenerima() + ", S.Kom.!";
    }
    // ... (dilewati)
}
```

Polimorfisme dari dalam Kelas: `self` dan `this`



`self/this` selalu menunjuk objek yang sebenarnya, bahkan ketika kode yang berjalan ditulis di kelas induk.

BAGIAN 4

4

Upcasting, *Downcasting*, dan *Duck Typing*

Memandang objek sebagai induk atau sebagai anak, lalu polimorfisme tanpa induk bersama.

Upcasting dan *Downcasting*

Upcasting: anak sebagai induk

- Objek `Mahasiswa` disimpan di variabel atau *list* bertipe `Pengguna`.
- Selalu **aman** dan terjadi **otomatis**: setiap mahasiswa pasti pengguna.
- Hanya boleh meminta yang bisa dilakukan semua pengguna, misalnya `peran()`.

Downcasting: induk sebagai anak

- Dibutuhkan untuk memanggil *method* khusus, misalnya `isi_krs()`/`isiKrs()`.
- **Tidak selalu aman**: belum tentu setiap pengguna adalah mahasiswa.
- Periksa dulu dengan `isinstance`/`instanceof`. Java menulis *cast*: `(Mahasiswa) pengguna`.

Memeriksa Jenis sebelum Memakai Fitur Khusus

Jenis objek diperiksa dulu, baru *method* khususnya dipanggil. Java boleh memakai *cast* atau bentuk singkat `instanceof Dosen dosen`.

Python python/contoh/contoh03_cek_jenis_objek.py

```python
for pengguna in daftar:
    if isinstance(pengguna, Mahasiswa):
        print(pengguna.isi_krs(21))
    elif isinstance(pengguna, Dosen):
        print(pengguna.input_nilai("PBO", "A"))
```

Java java/contoh/Contoh03UpcastingDowncasting.java

```java
for (Pengguna pengguna : daftar) {
    if (pengguna instanceof Mahasiswa) {
        Mahasiswa mhs = (Mahasiswa) pengguna;
        System.out.println(mhs.isiKrs(21));
    } else if (pengguna instanceof Dosen dosen) {
        System.out.println(dosen.inputNilai("PBO", "A"));
    }
}
```

Output

```
Ayu Lestari (2409106001) mengisi KRS 21 SKS
Rahmat Hidayat memasukkan nilai PBO: A
Bima Saputra (2409106002) mengisi KRS 21 SKS
```

Duck Typing: Yang Penting Bisa, Bukan Siapa

Situasi	Python	Java
Objek tanpa induk bersama, tetapi sama-sama punya `bunyikan()`	Bisa diproses dengan satu perulangan	Ditolak *compiler*; objek-objek itu butuh **tipe bersama**
Ada objek yang ternyata tidak punya `bunyikan()`	`AttributeError` saat program berjalan	Sudah ketahuan saat kompilasi
Kelebihan	Luwes dan ringkas	Kesalahan tertangkap lebih awal

Duck typing: objek dianggap cocok selama punya *method* yang dibutuhkan, walaupun tidak mewarisi induk yang sama.

BAGIAN 5

5

Overloading: Satu Nama, Parameter Berbeda

Bentuk polimorfisme yang dipilih saat kompilasi, dan bentuk ini khusus Java.

Overloading di Java

Overloading

Menulis beberapa *method* (atau konstruktor) dengan **nama yang sama** di kelas yang sama, tetapi **daftar parameternya berbeda**: jumlahnya, tipenya, atau urutan tipenya.

- Contoh: `cari(String)`, `cari(int)`, dan `cari(String, int)` di kelas `Perpustakaan`.
- Dipilih *compiler* saat kompilasi: **polimorfisme waktu kompilasi**. Tipe kembalian saja tidak cukup membedakan.
- Python **tidak punya** *overloading*: `def` kedua diam-diam menimpa `def` pertama.

Overriding dan *Overloading*

Aspek	*Overriding*	*Overloading*
Letak	Di kelas **anak**, menimpa *method* induk	Di kelas yang **sama** (atau ditambah di kelas anak)
Daftar parameter	**Sama**	**Berbeda**
Versi dipilih	Saat program berjalan, berdasarkan **tipe objek**	Saat kompilasi, berdasarkan **tipe argumen** yang tertulis
Tujuan	Mengganti perilaku untuk jenis yang lebih khusus	Menyediakan beberapa cara memanggil satu kemampuan
Python	Ada	Tidak ada; pakai nilai bawaan, argumen kata kunci, atau `*args`

STUDI KASUS

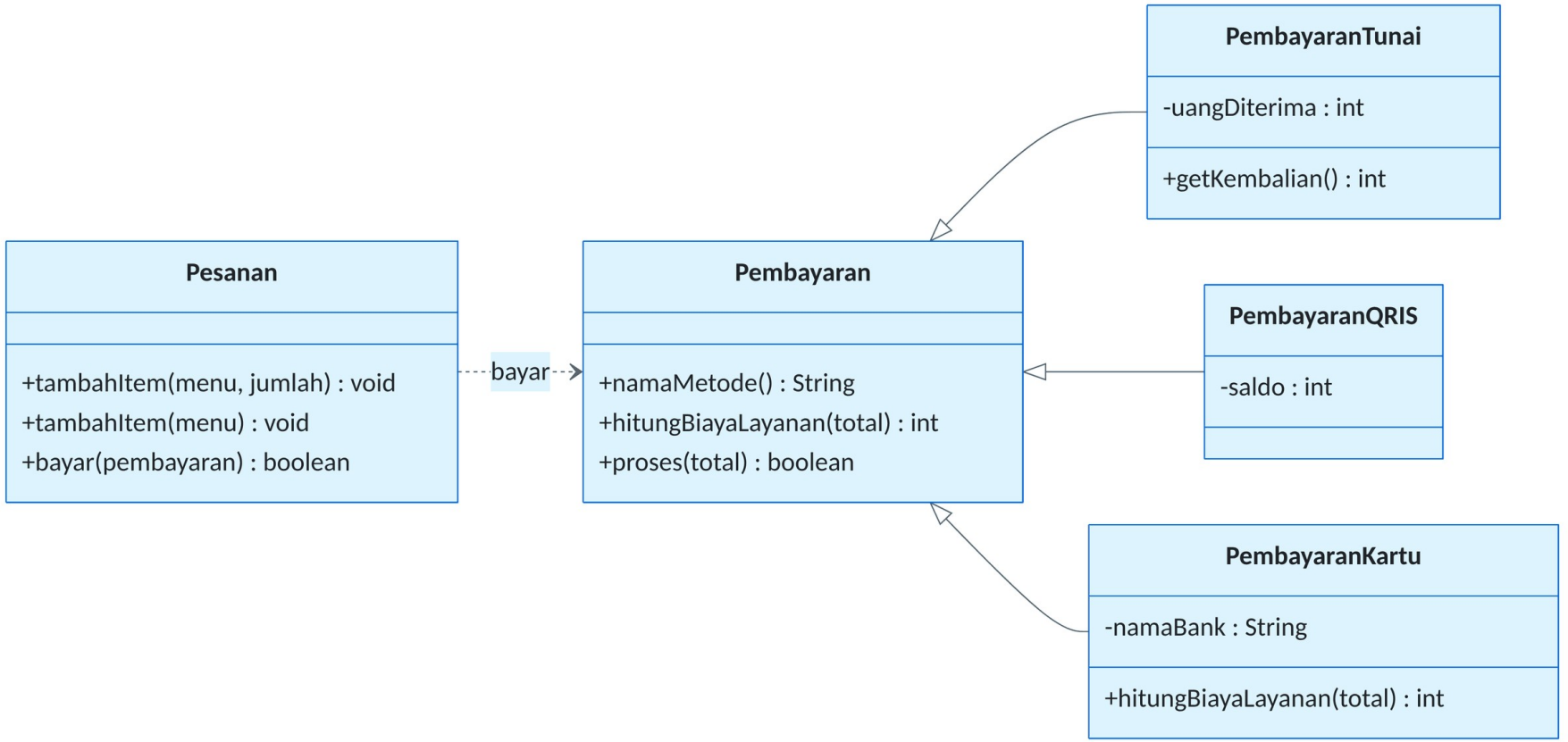


Studi Kasus KedaiKita

Satu *method* `bayar()` untuk tunai, QRIS, dan kartu kredit.

Rancangan: `Pembayaran` dan Tiga Kelas Anak

Studi kasus KedaiKita



Kotak anak hanya memuat tambahannya; ketiganya juga meng-*override* `namaMetode()` dan `proses(total)`. Nama bergaya Java.

`bayar()`: Tanpa Satu Pun Percabangan Jenis

Studi kasus KedaiKita

Biaya layanan, nama metode, dan proses dijawab oleh objek pembayaran yang sebenarnya. Tidak ada `isinstance`/`instanceof`.

Python python/studi-kasus/pesanan.py

```python
def bayar(self, pembayaran: Pembayaran) -> bool:
    """Membayar pesanan dengan metode pembayaran apa pun."""
    total = self.hitung_total()
    biaya = pembayaran.hitung_biaya_layanan(total)
    total_bayar = total + biaya
    # ... (dilewati)
    berhasil = pembayaran.proses(total_bayar)
    if berhasil:
        self._status = "LUNAS"
        # ... (dilewati)
    else:
        # ... (dilewati)
    return berhasil
```

Java java/studi-kasus/Pesanan.java

```java
public boolean bayar(Pembayaran pembayaran) {
    int total = hitungTotal();
    int biaya = pembayaran.hitungBiayaLayanan(total);
    int totalBayar = total + biaya;
    // ... (dilewati)
    boolean berhasil = pembayaran.proses(totalBayar);
    if (berhasil) {
        status = "LUNAS";
        // ... (dilewati)
    } else {
        // ... (dilewati)
    }
    return berhasil;
}
```

Pesan Satu Porsi: Nilai Bawaan dan *Overloading*

Studi kasus KedaiKita

Memanggil `tambah_item(menu)`/`tambahItem(menu)` tanpa jumlah berarti 1 porsi. Python memakai nilai bawaan, Java memakai *overloading*.

Python python/studi-kasus/pesanan.py

```python
def tambah_item(self, menu: Menu, jumlah: int = 1):
    item = ItemPesanan(menu, jumlah)
    menu.kurangi_stok(jumlah)
    self._daftar_item.append(item)
```

Java java/studi-kasus/Pesanan.java

```java
public void tambahItem(Menu menu, int jumlah) {
    ItemPesanan item = new ItemPesanan(menu, jumlah);
    menu.kurangiStok(jumlah);
    daftarItem.add(item);
}
// ... (dilewati)
public void tambahItem(Menu menu) {
    tambahItem(menu, 1);
}
```

Miskonsepsi yang Sering Terjadi

Anggapan keliru	Yang benar
“Polimorfisme butuh percabangan `if/else` untuk memilih versi *method* yang tepat.”	Justru sebaliknya: polimorfisme **menggantikan** percabangan berdasarkan jenis. Pemilihan versi dikerjakan otomatis oleh *dynamic dispatch*.
“*Casting* mengubah objek menjadi kelas lain.”	*Casting* hanya mengubah cara kita **memandang** objek. Objek `Dosen` tetap `Dosen` selamanya, jadi memaksanya dipandang sebagai `Mahasiswa` berakhir dengan `ClassCastException`.
“Python tidak punya polimorfisme karena tidak punya *overloading*.”	*Overloading* hanyalah satu bentuk polimorfisme. Python punya polimorfisme waktu berjalan (*override* + *dynamic dispatch*), bahkan lebih luwes lewat *duck typing*.

Kuis Cepat

Untuk *method* yang di-*override*, versi mana yang dijalankan ditentukan oleh ...

A. tipe variabel yang tertulis di kode

B. kelas yang paling dulu ditulis di dalam *file*

C. jumlah kelas anak yang dimiliki kelas induk

D. tipe objek yang sebenarnya, saat program berjalan

Kuis Cepat: Jawaban

Untuk *method* yang di-*override*, versi mana yang dijalankan ditentukan oleh ...

A tipe variabel yang tertulis di kode

B kelas yang paling dulu ditulis di dalam *file*

C jumlah kelas anak yang dimiliki kelas induk

✓ tipe objek yang sebenarnya, saat program berjalan

Jawaban: D

Inilah *dynamic dispatch*. Variabel bertipe `Hewan` yang menunjuk objek `Kucing` menjalankan `bersuara()` milik `Kucing`. Tipe variabel (pilihan A) hanya menentukan *method* apa saja yang **boleh dipanggil** di Java, bukan versi mana yang berjalan. Pilihan B dan C tidak berpengaruh sama sekali.

Kuis Cepat

Apa yang dimaksud dengan ***duck typing***?

A Java mengizinkan objek tanpa induk bersama disimpan dalam satu `List`

B Python menganggap objek cocok selama ia punya *method* yang dibutuhkan, tanpa memeriksa kelasnya

C Teknik memeriksa jenis objek dengan `isinstance` sebelum memanggil *method*

D Teknik menulis beberapa *method* bernama sama dengan parameter berbeda

Kuis Cepat: Jawaban

Apa yang dimaksud dengan ***duck typing***?

A. Java mengizinkan objek tanpa induk bersama disimpan dalam satu `List`

✓ Python menganggap objek cocok selama ia punya *method* yang dibutuhkan, tanpa memeriksa kelasnya

C. Teknik memeriksa jenis objek dengan `isinstance` sebelum memanggil *method*

D. Teknik menulis beberapa *method* bernama sama dengan parameter berbeda

Jawaban: B

“Jika berbunyi seperti bebek, anggap saja bebek.” Ayam jago, alarm, dan kentongan tanpa induk bersama tetap bisa diproses asalkan punya `bunyikan()`. Pilihan A keliru: Java justru butuh tipe bersama. Pilihan C adalah pemeriksaan jenis, kebalikan dari semangat *duck typing*. Pilihan D adalah *overloading*.

Ringkasan

- **Polimorfisme**: pemanggilan *method* yang sama ditanggapi dengan cara berbeda oleh objek yang berbeda.
- Polimorfisme waktu berjalan = ***override*** + ***dynamic dispatch***; versinya ditentukan **tipe objek**.
- **Koleksi polimorfik**: satu perulangan untuk banyak jenis; jenis baru cukup ditambahkan sebagai kelas baru.
- `self/this` selalu menunjuk objek sebenarnya, jadi *method* induk yang memanggil *method* lain juga polimorfik.
- ***Upcasting*** selalu aman. ***Downcasting*** harus didahului `isinstance/instanceof`.
- Python: ***duck typing***, tanpa *overloading*. Java: butuh tipe bersama dan punya ***overloading***.
- KedaiKita: `Pesanan.bayar()` menerima pembayaran jenis apa pun tanpa percabangan.

Langkah Berikutnya

- Jalankan contoh di halaman praktik (Python 01—06, Java 01—07) dan studi kasus KedaiKita.
- Kerjakan latihan, termasuk Latihan 5: bayar dengan *voucher*, lalu cek pemahamanmu dengan kuis 10 soal.

Buka bahan pertemuan ini di GitHub:

- Konsep
- Praktik Python
- Praktik Java
- Jembatan Python ↔ Java
- Latihan
- Kuis 10 soal

Pindai untuk membuka folder pertemuan ini di GitHub

github.com/antonprafanto/PBO2026/tree/master/Pertemuan-09-Polimorfisme